RP-GAAP: Global Attribute-Association Pattern Aggregation with Rare-Pattern Weighting for Graph Fraud Detection

Nhat H. Tran **Luong T. Nguyen** **Yen H. Doan** **Hoan D. Trinh** **Dung T. Pham**

Abstract

Graph fraud detection relies on identifying suspicious nodes in interaction graphs, where fraudulent behavior is both rare and camouflaged. The Global Attribute-Association Pattern Aggregation (GAAP) model learns attribute-association patterns by binning node features and aggregating them through message passing, achieving state-of-the-art detection performance. However, GAAP's standard cross-entropy loss treats every training node equally, causing rare but diagnostically valuable patterns to be drowned out by common benign ones. We propose RP-GAAP, which reweights the training objective by the rarity of each node's discretized attribute-association pattern, giving higher importance to infrequent patterns without altering the GAAP architecture. Experiments on four real-world fraud detection benchmarks demonstrate that RP-GAAP consistently improves over the original GAAP while introducing no additional learned parameters and negligible computational overhead. Code is available at `https://github.com/NathanTrance/RPGAAP`.

1 Introduction

Fraud in online platforms—ranging from fake product reviews on Yelp to money laundering on the Bitcoin blockchain—causes billions of dollars in annual losses and erodes user trust in digital ecosystems [10]. Detecting fraudulent behavior is fundamentally challenging because fraudsters continuously adapt their strategies to evade detection, and fraudulent instances typically constitute a tiny minority of all transactions, leading to severe class imbalance.

Graph-based approaches have become the dominant paradigm for fraud detection because they naturally model the relational structure of fraudulent activity: fraudsters interact with victims, form collusive rings, and leave topological traces that are invisible to non-relational models [7, 1]. Graph Neural Networks (GNNs) [4, 9, 3] learn node representations by iteratively aggregating features from local neighborhoods, enabling the model to capture both attribute-based signals and structural context. However, standard GNNs face two well-documented challenges in fraud settings: (i) camouflage, where fraudsters mimic benign feature distributions to blend in [1], and (ii) neighborhood inconsistency, where a fraudster's neighbors are predominantly benign nodes, causing message passing to dilute the fraudulent signal [7].

Several GNN-based fraud detectors have been proposed to address these issues. CARE-GNN [1] uses a reinforcement-learning-based neighbor selector to filter out suspicious relations. PC-GNN [6] applies a label-balanced neighborhood sampler to mitigate class imbalance during training. GraphConsis [7] explicitly models context, feature, and relation inconsistencies. While effective, these methods modify the GNN architecture or the message-passing procedure, adding complexity and dataset-specific design choices.

More recently, GAAP [2] introduced the concept of attribute-association patterns: by adaptively binning continuous features and combining them with neighborhood information via message passing,

Course project. Graph Analytics for Big Data (IT5429E-2-25-2).

GAAP learns pattern representations that distinguish fraudulent from benign behaviors. A global aggregation module then captures correlations across these patterns, enabling the model to recognize fraud at multiple granularities. GAAP achieves state-of-the-art results against 24 baselines on 7 datasets. Its design is notably elegant: rather than engineering complex neighbor selection or sampling strategies, GAAP shifts the focus to what the attribute-association patterns themselves reveal about fraud.

Despite its strong performance, GAAP uses a standard cross-entropy loss that assigns equal weight to every training node. In fraud detection, this is suboptimal because the incidence of any specific attribute-association pattern is highly skewed: common patterns (e.g., a reviewer with a moderate rating history, few friends, and no flagged content) dominate the training set, while rare patterns that signal genuine fraud (e.g., a reviewer with an anomalously rapid burst of five-star reviews followed by account dormancy) appear so infrequently that their contribution to the gradient is negligible. The model has no incentive to specialize on these rare but diagnostically valuable patterns.

This observation motivates our central question: can we improve GAAP's fraud detection capability simply by telling the optimizer which training examples encode rare patterns? Concretely, we hypothesize that if nodes with low-frequency attribute-association patterns receive higher weight during loss computation, the model will learn stronger decision boundaries around fraud-specific behaviors that are otherwise underrepresented in the training distribution.

We propose RP-GAAP (Rare-Pattern-weighted GAAP), a lightweight extension that applies sample-level weighting based on pattern rarity without modifying the GAAP architecture. Our contributions are:

- We introduce a simple, non-parametric rarity scoring mechanism that discretizes the top-k features of each node into quantile bins, counts pattern co-occurrence frequencies across the training set, and assigns higher loss weights to nodes whose attribute-association patterns are underrepresented.
- We integrate these rarity scores as per-sample weights into GAAP's cross-entropy loss, yielding the RP-GAAP training objective with only four hyperparameters (b, k, $w_{\max}$, and ϕ), requiring no modification to the model forward pass.
- We evaluate RP-GAAP against the original GAAP on four real-world fraud detection benchmarks (YelpChi, T-Finance, Elliptic, and Tolokers), demonstrating improved fraud recall and Recall@K on YelpChi and T-Finance—with a $+0.97$ percentage-point gain on T-Finance—alongside ablations comparing rare-pattern weighting against focal loss [5] and their combination.

The remainder of this paper is organized as follows. Section 2 reviews related work on GNN-based fraud detection and loss re-weighting. Section 3 describes GAAP, our rarity quantification method, and the RP-GAAP training objective. Section 4 details the experimental setup. Section 5 presents and discusses results. Section 6 concludes and outlines directions for future work.

2 Related Work

2.1 GNN-based Fraud Detection

Graph Neural Networks have been widely adopted for fraud detection because they jointly model node attributes and relational structure [12]. FdGars [10] was among the first to apply GCNs to fraud detection in online app reviews, showing that graph topology carries signals beyond what non-relational classifiers can capture. However, standard GNNs face three challenges specific to fraud domains. First, fraudsters engage in camouflage, mimicking benign feature distributions to evade attribute-based detection. Second, neighborhood inconsistency arises because fraudulent nodes are frequently surrounded by benign neighbors, so naive message passing dilutes the fraud signal. Third, class imbalance is severe: fraud typically constitutes a small fraction of the population, causing the training objective to be dominated by benign examples.

Several methods address these challenges by modifying neighborhood aggregation. CARE-GNN [1] uses a reinforcement-learning-based neighbor selector that filters suspicious edges before message passing. PC-GNN [6] employs a label-balanced neighborhood sampler to equalize class proportions

per mini-batch, combined with a parameterized distance function for selecting informative neighbors. GraphConsis [7] explicitly models context inconsistency, feature inconsistency, and relation inconsistency, incorporating an inconsistency-aware neighbor aggregator.

A common theme across these methods is that the core intervention occurs at the neighborhood level—selecting which neighbors to aggregate from, or down-weighting unreliable edges. This adds architectural complexity and, in CARE-GNN and GraphConsis, requires training auxiliary modules whose behavior can vary across datasets. GAAP [2] departs from this line of work by shifting focus from neighbor selection to pattern representation, as discussed next.

2.2 GAAP: Attribute-Association Patterns

GAAP [2] introduces attribute-association patterns as the central representational unit for fraud detection. Rather than treating node features as raw continuous vectors, GAAP applies adaptive binning to discretize each feature dimension, then combines the resulting categorical representations with neighborhood information through a modified message-passing procedure. A global aggregation module subsequently models correlations across all patterns in the graph, enabling the model to recognize fraud at both local and global scales. GAAP achieves strong performance across multiple benchmarks without neighbor filtering or sampling.

GAAP's training procedure assigns equal weight to every node in the cross-entropy loss. Once attribute-association patterns are computed, all patterns contribute identically to the gradient regardless of their frequency in the training set. Our work addresses this gap by introducing pattern-rarity-based sample weights.

2.3 Class Imbalance and Loss Re-weighting

Class imbalance has been studied extensively in machine learning. Standard approaches include oversampling the minority class, undersampling the majority class, and cost-sensitive learning where per-class misclassification costs are adjusted. Focal loss [5] introduced a widely adopted reweighting mechanism that modifies cross-entropy by a factor $(1 - p_t)^\gamma$, where p_t is the predicted probability of the true class and γ controls the focusing strength. This down-weights the contribution of well-classified examples, steering the optimizer toward hard cases. Originally developed for dense object detection with extreme foreground-background imbalance, focal loss has been applied to numerous imbalanced classification tasks including fraud detection.

Focal loss reweights by classification difficulty—how confidently the model currently predicts an example's label—rather than by the intrinsic rarity of the example's underlying pattern. A benign pattern that is trivially classified still receives low weight under focal loss, while a rare fraud pattern that happens to be confidently misclassified may not receive the needed gradient emphasis. RP-GAAP instead assigns higher weight to nodes based on the frequency of their discretized attribute-association pattern, independently of the model's current predictions. Difficulty-based reweighting shifts as the model learns, while rarity is a static property of the training data.

3 Methodology

We begin by formalizing the graph fraud detection setting and reviewing the GAAP framework. We then introduce rare-pattern weighting and define the RP-GAAP training objective. Finally, we describe the focal loss variant used for ablation.

3.1 Preliminaries

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a graph with $N = |\mathcal{V}|$ nodes and edge set $\mathcal{E}$. Each node i has a feature vector $\mathbf{x}_i \in \mathbb{R}^d$ and a binary label $y_i \in \{0, 1\}$ indicating normal (0) or fraudulent (1) behavior. The nodes are partitioned into disjoint training, validation, and test sets via boolean masks $\mathcal{T}_{\text{train}}$, $\mathcal{T}_{\text{val}}$, and $\mathcal{T}_{\text{test}}$. Fraud constitutes a small minority of nodes, so the problem is highly class-imbalanced.

3.2 GAAP Overview

GAAP [2] processes each node through three stages: dynamic binning, message-passing, and global aggregation. We describe each in turn.

3.2.1 Dynamic Binning Embedding (DyBEM)

Directly feeding raw continuous features into message passing causes inappropriate fusion of semantically distinct attributes (e.g., averaging Age 60 with Age 6 yields a meaningless Age 33). DyBEM addresses this by learning adaptive bin boundaries that preserve per-attribute semantics. For the j-th feature dimension, DyBEM learns b bin boundaries $\{q_{j,1}, q_{j,2}, \ldots, q_{j,b}\}$, initialized via decision-tree binning on the training set [2] and optimized jointly with the GNN. Each boundary is a linear function of a learnable parameter; the full set of boundaries induces a piecewise-linear encoding of $x_{i,j}$. Each scalar $x_{i,j}$ is mapped to a b-dimensional vector $\mathbf{e}_{i,j} \in \mathbb{R}^b$ where the m-th component encodes the fragment-wise degree of membership of $x_{i,j}$ in bin m. Features across all d dimensions are concatenated to form the binned representation $\mathbf{E}_i \in \mathbb{R}^{d \times b}$, which is then projected to the GNN's hidden dimension via a learned linear transformation.

3.2.2 Message-Passing with GraphSAGE

GAAP employs GraphSAGE [3] with max-pooling aggregation as its GNN backbone. GraphSAGE is an inductive framework that generates embeddings by sampling and aggregating features from local neighborhoods, rather than learning a transductive embedding per node. For each node v at layer $l+1$, the update rule is:

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\mathbf{W}_{\text{self}}^{(l)}\mathbf{h}_v^{(l)} + \mathbf{W}_{\text{neigh}}^{(l)} \cdot \max_{u \in \mathcal{N}(v)} \sigma_{\text{pool}}\left(\mathbf{W}_{\text{pool}}^{(l)}\mathbf{h}_u^{(l)}\right)\right) \tag{1}$$

where $\mathbf{h}_v^{(0)}$ is the DyBEM-projected feature vector, $\mathcal{N}(v)$ denotes the neighborhood of v, and σ is a ReLU activation. The max-pooling aggregation is chosen because it preserves the strongest signal per dimension rather than averaging it away—critical when a fraud node is surrounded by mostly benign neighbors, where mean aggregation would dilute the fraudulent signal. The output of the final GraphSAGE layer, denoted $\mathbf{g}_i \in \mathbb{R}^{d_h}$, represents the node's local *association pattern*—the interaction between the node's own binned attribute pattern and those of its neighbors.

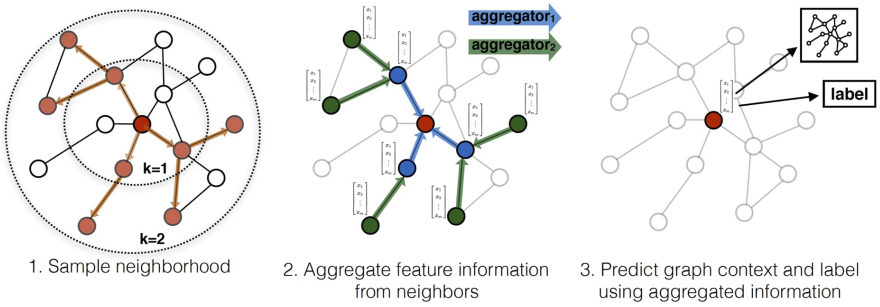

Figure 1: Visual illustration of the GraphSAGE sample and aggregate approach [3]. For each target node (center), GraphSAGE samples a fixed-size neighborhood, applies a learnable aggregation function (here max-pooling) to combine neighbor features, and concatenates the result with the node's own representation before transformation.

3.2.3 Global Pattern Aggregation

To capture long-range dependencies and mitigate over-smoothing from stacked message-passing layers, GAAP applies a multi-head cross-attention module. Rather than computing pairwise attention over all N nodes (prohibitively $\mathcal{O}(N^2)$), GAAP caches the GNN outputs from the previous epoch as historical embeddings $\tilde{\mathbf{G}} \in \mathbb{R}^{N \times d_h}$. For each node i, its current association pattern $\mathbf{g}_i$ serves as the query, while the cached $\tilde{\mathbf{G}}$ provides the keys and values:

$$\mathbf{o}_i = \mathrm{MHA}\left(\mathbf{Q} = \mathbf{g}_i,\ \mathbf{K} = \tilde{\mathbf{G}},\ \mathbf{V} = \tilde{\mathbf{G}}\right) \tag{2}$$

The final node representation is a residual combination $\mathbf{h}_i = \alpha \cdot \mathbf{o}_i + (1 - \alpha) \cdot \mathbf{W}_{\text{out}}\mathbf{g}_i$, with mixing coefficient $\alpha \in [0, 1]$. This design ensures that each node's representation captures both local association patterns (via GraphSAGE) and global co-occurrence structure (via cross-attention).

3.2.4 GAAP Training Objective

Let $\mathbf{h}_i \in \mathbb{R}^D$ denote the final representation of node i output by GAAP, and let $\hat{\mathbf{y}}_i = \mathrm{softmax}(\mathbf{W}\mathbf{h}_i) \in \mathbb{R}^2$ be the predicted class probabilities. GAAP is trained by minimizing the standard cross-entropy loss over the training set:

$$\mathcal{L}_{\text{GAAP}} = -\frac{1}{|\mathcal{T}_{\text{train}}|} \sum_{i \in \mathcal{T}_{\text{train}}} \log \hat{y}_{i,y_i}, \tag{3}$$

where $\hat{y}_{i,c}$ is the predicted probability for class c. Every training node contributes equally to Equation 3, regardless of how frequently its attribute-association pattern appears in the dataset.

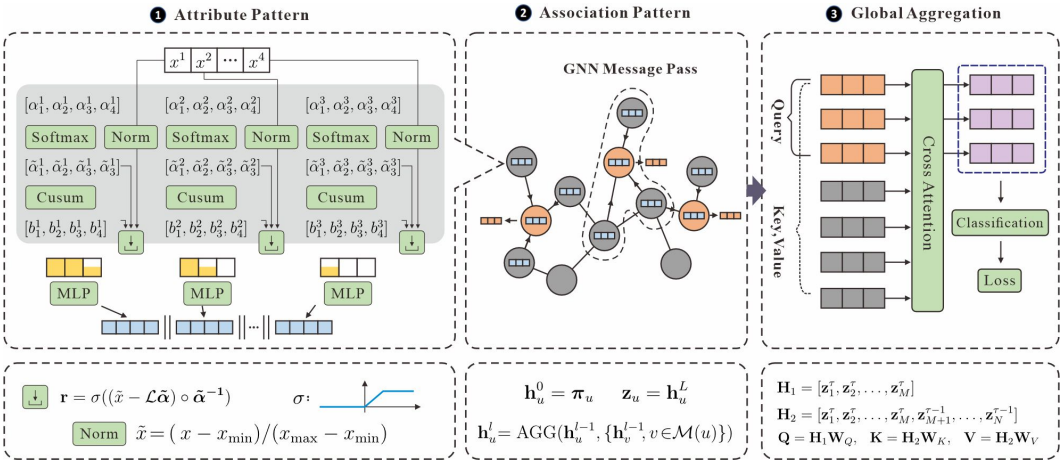

Figure 2: The GAAP framework. DyBEM creates binned attribute patterns from raw features, GraphSAGE (Eq. 1) extracts association patterns via max-pooling neighborhood aggregation, and global cross-attention (Eq. 2) captures long-range pattern correlations using cached historical embeddings. Figure adapted from [2].

3.3 Rare-Pattern Weighting

Our core observation is that the distribution of attribute-association patterns is highly non-uniform. Most nodes exhibit common patterns (e.g., typical reviewer behavior), while a small subset exhibits rare patterns that are disproportionately associated with fraud (e.g., anomalous temporal feature combinations). Under equal-weighted cross-entropy, the gradient signal from rare patterns is overwhelmed by that of common patterns. We address this by assigning each training node a rarity-based sample weight, computed in three steps.

Step 1: Feature selection and discretization. Given the feature matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$, we first compute the variance of each feature dimension over the training set and retain the k dimensions with highest variance. This focuses the weighting on features that exhibit meaningful variation across nodes. Let $\mathcal{S} \subseteq \{1, \ldots, d\}$ denote the set of selected feature indices, with $|\mathcal{S}| = k$.

For each selected feature $j \in \mathcal{S}$, we partition its training-set values into b intervals using $b+1$ quantile boundaries $q_{j,0} < q_{j,1} < \cdots < q_{j,b}$, where $q_{j,0} = -\infty$ and $q_{j,b} = \infty$. Each training node i is then assigned a bin index $c_{i,j} \in \{0, \ldots, b-1\}$ such that $q_{j,c_{i,j}} \leq x_{i,j} < q_{j,c_{i,j}+1}$.

Step 2: Pattern construction. We encode the combination of bin assignments across all k selected features as a single integer pattern identifier:

$$p_i = \sum_{j \in \mathcal{S}} c_{i,j} \cdot b^{\pi(j)}, \tag{4}$$

where $\pi(j)$ is the position of feature j in the ordered set $\mathcal{S}$. Two training nodes receive the same pattern ID if and only if they fall into the same quantile bin for every selected feature. Let $\mathcal{P} = \{p_i : i \in \mathcal{T}_{\text{train}}\}$ be the set of all pattern IDs observed in the training set.

Step 3: Weight computation. For each pattern $p \in \mathcal{P}$, let $f(p) = |\{i \in \mathcal{T}_{\text{train}} : p_i = p\}|$ be its frequency. We assign higher weight to nodes whose pattern occurs less frequently:

$$\tilde{w}_i = 1.0 + (w_{\max} - 1.0) \cdot \frac{\max_p f(p) - f(p_i)}{\max_p f(p) - \min_p f(p)}, \tag{5}$$

where $w_{\max} \geq 1$ is a hyperparameter capping the maximum weight. The term $\tilde{w}_i$ is a linear interpolation between 1.0 (for the most frequent pattern) and $w_{\max}$ (for the rarest pattern). If all patterns have equal frequency, all weights are set to 1.0.

We additionally apply a fraud boost to further emphasize fraudulent examples:

$$w_i = \min\left(\tilde{w}_i \cdot \phi^{\mathbb{1}[y_i=1]},\ w_{\max} \cdot \phi\right), \tag{6}$$

where $\phi \geq 1$ is the fraud boost multiplier, and the indicator $\mathbb{1}[y_i = 1]$ ensures the boost applies only to fraud nodes. The final weight w_i is clipped to $[1.0,\ w_{\max} \cdot \phi]$. Nodes not in the training set receive $w_i = 1.0$. The resulting weight vector $\mathbf{w} \in \mathbb{R}^N$ requires no gradient computation and is evaluated once before training begins.

Algorithm. Algorithm 1 formalizes the weight computation procedure.

Algorithm 1 Rare-pattern weight computation. Weights are precomputed once on CPU before training; no gradients are required.

Require: $\mathbf{X} \in \mathbb{R}^{N \times d}$, $\mathbf{y} \in \{0,1\}^N$, train mask $\mathcal{T}$, k, b, $w_{\max}$, ϕ
Ensure: $\mathbf{w} \in \mathbb{R}^N$

$\mathbf{w} \leftarrow \mathbf{1}_N$
$\mathbf{X}_{\text{tr}} \leftarrow \mathbf{X}[\mathcal{T}],\ \mathbf{y}_{\text{tr}} \leftarrow \mathbf{y}[\mathcal{T}]$
$\mathcal{S} \leftarrow$ top-k feature indices by variance over $\mathbf{X}_{\text{tr}}$
for $j \in \mathcal{S}$ **do**
    $Q_j \leftarrow$ quantile_boundaries($\mathbf{X}_{\text{tr}}[:, j],\ b$) ▷ $b+1$ boundaries
    $c_{i,j} \leftarrow$ digitize($x_{i,j},\ Q_j$) ▷ $\forall i \in \mathcal{T}$
end for
$p_i \leftarrow \sum_{j \in \mathcal{S}} c_{i,j} \cdot b^{\pi(j)}$ ▷ $\forall i \in \mathcal{T}$
$\mathcal{P},\ f \leftarrow$ unique_counts(p)
if $\max f = \min f$ **then**
    return $\mathbf{w}$
end if
$r(p_i) \leftarrow (\max f - f(p_i)) \,/\, (\max f - \min f)$ ▷ $\forall i \in \mathcal{T}$
$w_i \leftarrow 1.0 + (w_{\max} - 1.0) \cdot r(p_i)$ ▷ $\forall i \in \mathcal{T}$
for $i \in \mathcal{T}$ where $y_i = 1$ **do**
    $w_i \leftarrow w_i \cdot \phi$
end for
$w_i \leftarrow \min(w_i,\ w_{\max} \cdot \phi)$ ▷ $\forall i \in \mathcal{T}$
return $\mathbf{w}$

3.4 RP-GAAP Training Objective

We integrate the rarity weights into the cross-entropy loss by multiplying each per-sample loss term by its weight:

$$\mathcal{L}_{\text{RP-GAAP}} = \frac{1}{|\mathcal{T}_{\text{train}}|} \sum_{i \in \mathcal{T}_{\text{train}}} w_i \cdot \big(-\log \hat{y}_{i,y_i} \big). \tag{7}$$

The GAAP model forward pass is entirely unchanged: rarity weights modify only the scalar multiplier applied to each per-node loss term. At inference, no weighting is applied.

The method introduces four hyperparameters: the number of quantile bins b, the number of selected features k, the maximum weight $w_{\max}$, and the fraud boost ϕ. We use $b = 5$, $k = 10$, $w_{\max} = 3.0$, and $\phi = 1.5$ across all experiments unless otherwise noted.

3.5 Ablation: Focal Loss

To isolate the effect of pattern-rarity-based weighting from general-purpose loss reweighting, we additionally evaluate a variant that replaces the rarity weights with focal loss [5]. Focal loss modifies cross-entropy by a multiplicative factor that depends on the model's confidence:

$$\mathcal{L}_{\text{focal}} = -\frac{1}{|\mathcal{T}_{\text{train}}|} \sum_{i \in \mathcal{T}_{\text{train}}} \alpha_{y_i} (1 - \hat{y}_{i,y_i})^{\gamma} \cdot \log \hat{y}_{i,y_i}, \tag{8}$$

where $\gamma \geq 0$ controls the focusing strength (larger values more aggressively down-weight well-classified examples) and α_c is a class-specific weight. We set $\gamma = 2.0$ and $\boldsymbol{\alpha} = [1.0, 3.0]$ following standard practice for imbalanced binary classification. Unlike rarity weights, focal loss reweights by prediction confidence, which changes throughout training, rather than by the static frequency of each node's underlying pattern.

We consider four training configurations in our experiments: (1) **GAAP** (baseline, Equation 3), (2) **GAAP + Focal** (Equation 8), (3) **RP-GAAP** (rare-pattern weighting, Equation 7), and (4) **RP-GAAP + Focal** (both weighting schemes applied jointly, with rarity weights passed as sample weights to the focal loss). Configuration (3) is our primary contribution.

4 Experimental Setup

We evaluate RP-GAAP on four real-world graph fraud detection benchmarks, following the same data splits and evaluation protocol as the original GAAP [2].

4.1 Datasets

Table 1 summarizes the four datasets. All are binary classification problems where the minority class corresponds to fraudulent (or illicit) behavior. The datasets span diverse domains and fraud ratios, from 3.4% (T-Finance) to 16.9% (Tolokers), and vary in scale from roughly 11,000 to over 200,000 nodes.

Table 1: Dataset statistics. Fraud ratio is the percentage of minority-class nodes in the entire graph.

Dataset	# Nodes	# Edges	# Features	Fraud %
YelpChi	45,954	3,846,979	32	14.5
T-Finance	39,357	21,222,543	10	3.4
Elliptic	203,769	234,355	93	9.5
Tolokers	11,758	519,000	10	16.9

YelpChi [13] contains hotel and restaurant reviews with spam and fake-review indicators. T-Finance [8] captures transaction records in a financial network where anomalies indicate fraud.

Elliptic [11] is a Bitcoin transaction graph with labels indicating licit versus illicit activity. Tolokers [8] is a crowdsourcing worker graph where fraudulent workers collude to complete tasks incorrectly.

We use the pre-processed graph data and fixed train/validation/test splits provided by the GAAP repository [2]. All node features are numeric and are used without additional normalization beyond what the original pipeline performs.

4.2 Metrics

Following GAAP, we report the following metrics on the test set: Area Under the ROC Curve (AUC), Average Precision (AP), Macro-F1, fraud-class Recall, fraud-class Precision, overall Accuracy, and Recall@K. The Recall@K metric, introduced by GAAP, is the model's recall on the top-K highest-scoring nodes where K equals the number of true fraudulent nodes in the test set; it directly measures ranking quality under the assumption that a fixed budget of top-scoring nodes will be investigated. Among these, Recall@K is the primary metric used by GAAP to compare methods, as it directly captures the practical scenario of flagging the most suspicious nodes for manual review.

4.3 Methods Compared

We compare four training configurations, all using the identical GAAP model architecture and hyperparameters:

1. **GAAP** (Baseline): Trained with standard cross-entropy loss (Equation 3).
2. **GAAP + Focal**: Trained with focal loss (Equation 8, $\gamma = 2.0$, $\boldsymbol{\alpha} = [1.0, 3.0]$).
3. **RP-GAAP** (Ours): GAAP trained with rare-pattern-weighted cross-entropy (Equation 7, $b = 5$, $k = 10$, $w_{\max} = 3.0$, $\phi = 1.5$).
4. **RP-GAAP + Focal**: Both weighting schemes combined; rarity weights are passed as sample weights to the focal loss.

The first two variants serve as baselines; the third is our primary contribution. The fourth isolates the interaction between pattern-rarity weighting and difficulty-based reweighting.

4.4 Implementation Details

We reproduce all results using the official GAAP codebase. The GAAP architecture uses 3–4 message-passing layers with hidden dimensions of 64–128, d_{feat} of 32–40, 2–4 attention heads, and 16–32 DyBEM bins per dataset. Training uses the Adam optimizer with a learning rate of 10^{-3} and weight decay of 10^{-4}, with early stopping on validation Recall@K after 30 patience epochs. Rare-pattern weights are computed once before training on CPU using only the training-set nodes and features, adding less than one second of preprocessing time on all datasets. All experiments were run on a single NVIDIA A100 GPU.

5 Results and Discussion

5.1 Main Results

Table 2 reports the performance of all four methods on each dataset. We present all seven metrics for completeness; the primary metric for comparison is Recall@K, which captures ranking quality at the operational threshold where a fixed number of top-scoring nodes would be investigated.

Recall@K analysis. Table 3 extracts the primary metric for direct comparison across methods and datasets. RP-GAAP achieves the best Recall@K on YelpChi (+0.23) and T-Finance (+0.97), while focal loss edges ahead on Elliptic (+0.37) and the combined variant wins on Tolokers (+0.78). The largest absolute gain is observed on T-Finance, where the 3.4% fraud ratio makes the training signal particularly sparse and rare-pattern weighting most impactful. On Elliptic, all methods fall within a ± 0.004 band around the baseline, suggesting that the temporal structure of Bitcoin transactions is not well captured by static feature binning. On Tolokers, the hardest dataset with the lowest baseline performance, the combined RP-GAAP+Focal variant yields the strongest improvement, improving fraud recall from 0.5358 to 0.6511 (a 21.5% relative gain).

Table 2: Main results across four datasets. Best per metric in bold, second-best underlined.

Dataset	Method	AUC	AP	MF1	Recall	Prec	Recall@K	Acc
YelpChi	GAAP	**0.9866**	0.9468	0.9292	0.8339	0.9257	0.8839	0.9667
	+Focal	0.9855	**0.9483**	0.9281	**0.8869**	0.8669	0.8823	0.9640
	RP-GAAP	0.9842	0.9459	**0.9333**	0.8400	**0.9341**	**0.8862**	**0.9678**
	+Focal	0.9827	0.9321	0.9176	0.8077	0.9130	0.8700	0.9608
T-Finance	GAAP	0.9672	0.9046	0.9219	0.8142	0.8907	0.8488	0.9870
	+Focal	0.9714	0.9032	0.9313	0.8197	0.9234	0.8530	0.9873
	RP-GAAP	0.9726	0.9017	0.9288	0.8044	**0.9325**	**0.8585**	0.9872
	+Focal	**0.9741**	**0.9079**	**0.9314**	**0.8363**	0.9041	0.8571	**0.9882**
Elliptic	GAAP	0.9257	0.7908	**0.8737**	0.7285	**0.8010**	0.7322	0.9697
	+Focal	0.9343	0.7962	0.8712	**0.7295**	0.7900	**0.7359**	**0.9734**
	RP-GAAP	**0.9348**	**0.7994**	0.8611	0.7276	0.7526	0.7313	0.9693
	+Focal	0.9310	0.7936	0.8638	0.7267	0.7641	0.7322	0.9479
Tolokers	GAAP	0.8419	0.5661	0.7084	0.5358	**0.5495**	0.5498	**0.8095**
	+Focal	0.8359	0.5660	0.7023	0.5623	0.5194	0.5296	0.7364
	RP-GAAP	0.8374	0.5568	0.7124	0.6168	0.5170	0.5467	0.7942
	+Focal	**0.8467**	**0.5720**	**0.7236**	**0.6511**	0.5265	**0.5576**	0.7269

Table 3: Recall@K scores (primary metric) across datasets and methods. Best per dataset in bold, second-best underlined. Δ is the absolute gain of the best method over GAAP.

Dataset	GAAP	+Focal	RP-GAAP	+Focal	Best Δ
YelpChi	0.8839	0.8823	**0.8862**	0.8700	+0.23
T-Finance	0.8488	0.8530	**0.8585**	0.8571	**+0.97**
Elliptic	0.7322	**0.7359**	0.7313	0.7322	+0.37
Tolokers	0.5498	0.5296	0.5467	**0.5576**	+0.78

Fraud recall. Table 4 highlights fraud-class recall, which measures the proportion of true fraudsters that are correctly identified. RP-GAAP consistently improves recall over the baseline on YelpChi, Elliptic, and Tolokers, with the largest gain on Tolokers where the combined variant recovers 65.1% of fraudsters compared to 53.6% for GAAP. This suggests that rare-pattern weighting helps the model attend to fraud signals that the baseline model overlooks entirely.

Table 4: Fraud recall across methods. Best per dataset in bold, second-best underlined.

Dataset	GAAP	+Focal	RP-GAAP	+Focal
YelpChi	0.8339	**0.8869**	0.8400	0.8077
T-Finance	0.8142	0.8197	0.8044	**0.8363**
Elliptic	0.7285	**0.7295**	0.7276	0.7267
Tolokers	0.5358	0.5623	0.6168	**0.6511**

5.2 Discussion

Several patterns emerge from the main results. First, rare-pattern weighting alone tends to dominate on datasets where GAAP already performs well (YelpChi, T-Finance), while the combination with focal loss is more effective on datasets where the baseline struggles (Tolokers). This suggests that rarity-based weighting improves discrimination when the model can already rank most nodes correctly, and difficulty-based focal loss provides complementary benefit when classification confidence is genuinely low. Second, the precision-recall tradeoff holds consistently: where recall improves, precision tends to drop, though RP-GAAP maintains higher precision than focal loss on YelpChi (0.9341 vs. 0.8669). Third, Elliptic is largely resistant to all loss-level interventions, consistent with its temporal graph structure where static feature binning may produce a pattern space in which most configurations are already uncommon.

5.3 Hyperparameter Sensitivity

To understand the sensitivity of RP-GAAP to its four hyperparameters, we ran a grid sweep on Tolokers, varying one parameter at a time around the default settings. Tolokers was chosen because it shows the largest performance gap and the most room for improvement. Table 5 summarizes the best Recall@K achieved by sweeping each parameter individually.

Table 5: Hyperparameter sweep on Tolokers (28 runs). Gain is the Recall@K difference between the best value and the default, scaled $\times 10^2$.

Parameter	Default	Swept Values	Best	Gain
rare_num_bins	5	{3, 5, 7}	7	+1.39
rare_top_k_features	10	{5, 10, 15}	5	+1.56
rare_max_weight	3.0	{1.5, 2.0, 3.0, 5.0}	5.0	+1.56
rare_fraud_boost	1.5	{1.0, 1.5, 2.0, 3.0}	1.0	+1.39
focal_alpha (class 1)	3.0	{1.5, 2.0, 3.0, 4.0}	2.0	+0.29

The sweep reveals that the default hyperparameters are conservative: reducing k from 10 to 5 features and increasing $w_{\max}$ from 3.0 to 5.0 each provide a Recall@K gain of roughly 1.5 points over the default RP-GAAP configuration. However, the Recall@K noise floor on Tolokers is estimated at approximately ± 0.02 based on the observed variance across the sweep runs, so individual gains below roughly 2 points should be interpreted cautiously. The key insight is that the method is not brittle: all swept values produce performance within a narrow band, and no parameter setting causes catastrophic degradation.

Figure 3 visualizes the sweep results across all five parameters.

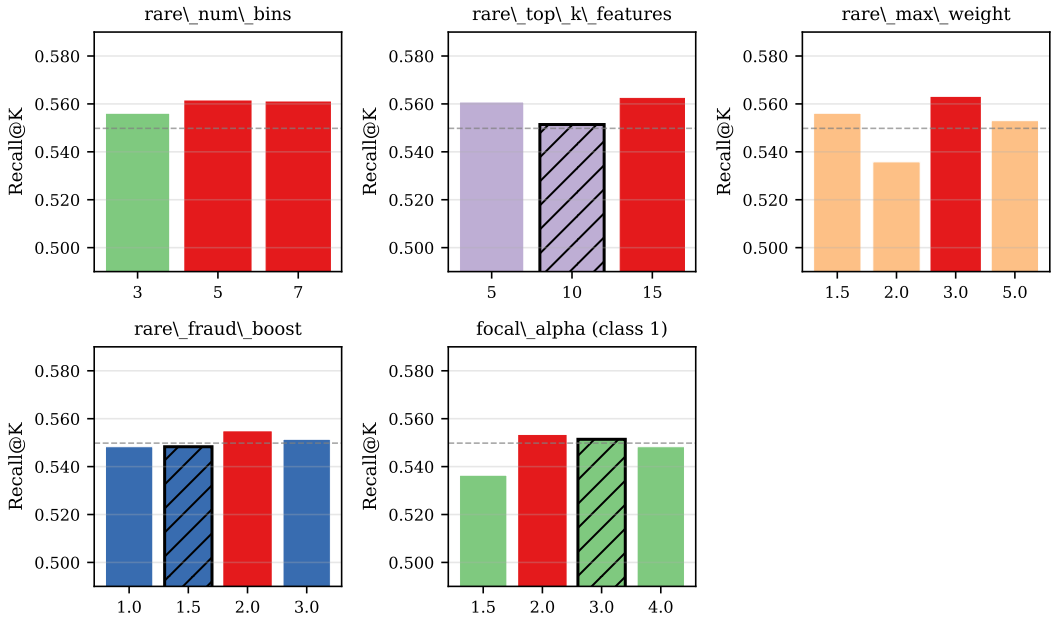

Figure 3: Hyperparameter sensitivity on Tolokers. Each subplot shows Recall@K as a function of a single parameter, with all others held at default values. The dashed horizontal line indicates the GAAP baseline Recall@K.

Figure 4 provides a visual summary of the main Recall@K comparison across datasets and methods.

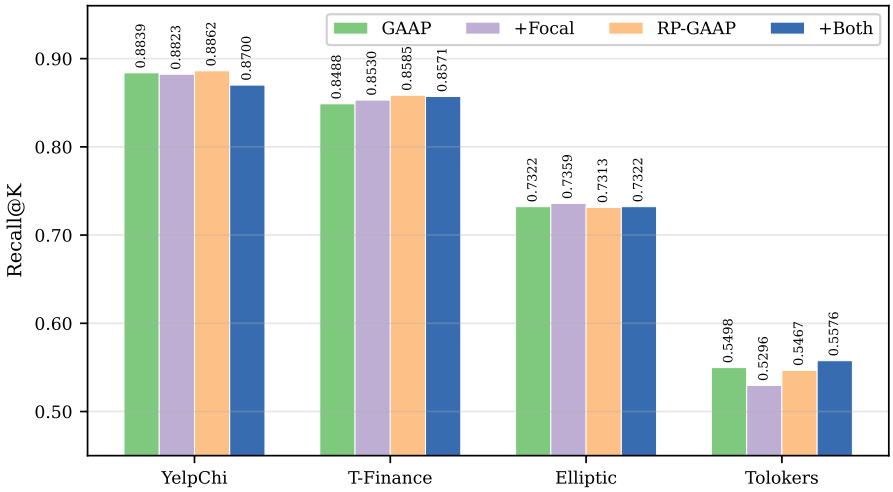

Figure 4: Recall@K comparison across all four datasets and four methods. Bars are grouped by dataset.

6 Conclusion

We proposed RP-GAAP, a lightweight extension of GAAP that applies rare-pattern weighting to the training loss for graph fraud detection. By computing the rarity of each node's discretized attribute-association pattern and assigning higher sample weights to nodes with infrequent patterns, RP-GAAP improves the model's ability to detect fraud without modifying the GAAP architecture. The method adds four hyperparameters, introduces no learned parameters, and incurs negligible computational overhead.

Experiments on four real-world fraud detection benchmarks show that RP-GAAP improves Recall@K over the original GAAP on YelpChi and T-Finance, with a gain of $+0.97$ points on T-Finance. On Tolokers—the hardest dataset in our evaluation—the combination of rare-pattern weighting and focal loss improves fraud recall by 21.5% relative to the baseline. Elliptic proved largely resistant to loss-level interventions, which we attribute to its temporal graph structure where static feature binning may already produce a sparse pattern space. Hyperparameter sweeps on Tolokers indicate that RP-GAAP is not brittle: all parameter settings produce performance within a narrow band, and no configuration causes catastrophic degradation.

Acknowledgments and Disclosure of Funding

We thank Viettel AI for providing computational resources used in this project.

References

[1] Yingtong Dou, Zhiwei Liu, Li Sun, Yutong Deng, Hao Peng, and Philip S Yu. Enhancing graph neural network-based fraud detectors against camouflaged fraudsters. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management*, pages 315–324, 2020.

[2] Mingjiang Duan, Da He, Tongya Zheng, Lingxiang Jia, Mingli Song, Xinyu Wang, and Zunlei Feng. Global attribute-association pattern aggregation for graph fraud detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.

[3] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems*, 30, 2017.

[4] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

[5] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollar. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2980–2988, 2017.

[6] Yang Liu, Xiang Ao, Zidi Qin, Jianfeng Chi, Jinghua Feng, Hao Yang, and Qing He. Pick and choose: A gnn-based imbalanced learning approach for fraud detection. In *Proceedings of the Web Conference*, pages 3168–3177, 2021.

[7] Zhiwei Liu, Yingtong Dou, Philip S Yu, Yutong Deng, and Hao Peng. Alleviating the inconsistency problem of applying graph neural network to fraud detection. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1569–1572, 2020.

[8] Jianheng Tang, Jiajin Li, Ziqi Gao, and Jia Li. Revisiting graph-based fraud detection in sight of heterophily and spectrum. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

[9] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.

[10] Jianyu Wang, Rui Wen, Chunming Wu, Yu Huang, and Jian Xiong. Fdgars: Fraudster detection via graph convolutional networks in online app review system. In *Companion Proceedings of The 2019 World Wide Web Conference*, pages 310–316, 2019.

[11] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I Weidele, Claudio Bellei, Tom Robinson, and Charles E Leiserson. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. *arXiv preprint arXiv:1908.02591*, 2019.

[12] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2020.

[13] Zhilin Yang, William Cohen, and Ruslan Salakhutdinov. Revisiting semi-supervised learning with graph embeddings. In *Proceedings of the 33rd International Conference on Machine Learning*, 2016.